Program-1

Feed Forward Neural Network for MNIST Digit Classification

Aim: Implement multi-layer perceptron algorithm for MNIST Handwritten Digit Classification.

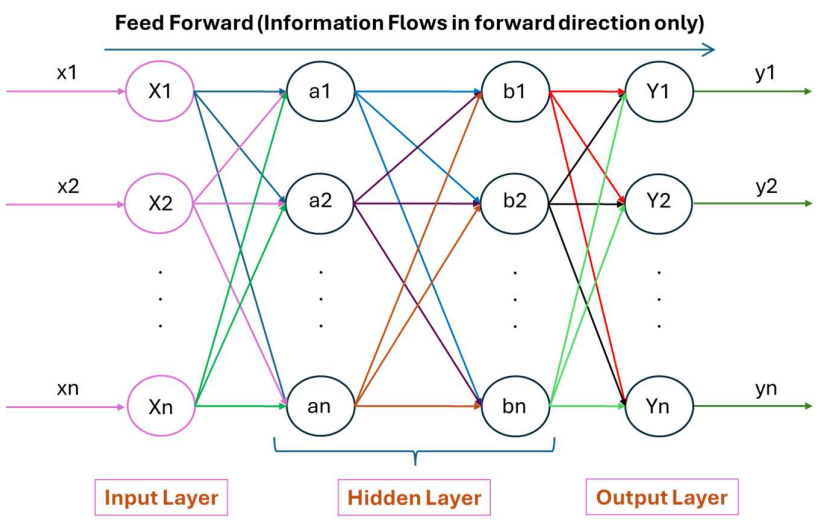
Introduction to MNIST Dataset:

The MNIST (Modified National Institute of Standards and Technology) dataset is a very popular dataset used in the field of computer vision and deep learning. Because of its simplicity and widespread use for introducing and testing new algorithms.

The MNIST dataset is a large collection of **handwritten digits**. The digits are standardized to a fixed size and are represented as grayscale images.

- **Size**: The dataset contains 70,000 images in total.
  - **Training Set**: 60,000 images for training a model.
  - **Test Set**: 10,000 images for evaluating the trained model.
- **Image Dimensions**: Each image is a **28x28 pixel grayscale image**. Since it's grayscale, each pixel has a single value representing its intensity, typically ranging from 0 (black) to 255 (white).
- **Labels**: Each image has a corresponding label, which is an integer from 0 to 9, indicating the digit it represents.

Introduction to Feed Forward Neural Networks:



The information flows from input layer to the output layer, via hidden layers. The input layer receives the input from the sources and supplies to the neurons in the hidden layer-1. The hidden layer performs computations at every neuron and transmits the information to hidden layer-2. Again, computations are performed at every neuron and transmits the information to hidden layer-3. This process continues until the information passes to the final hidden layer. Later the information passes to the output layer in a form suitable for representation as output. In this process, activation functions are applied at various layers based on the type of problem to be solved. The number of hidden layers, number of neurons in each layer and the activation functions can be varied for improving the performance of the neural network.

Applications for Feed Forward Neural Network:

1. **General Classification Tasks**: FNNs are widely used to classify data into predefined categories. The network takes input features and outputs a prediction for the most likely class.
   - **Image Recognition**: For simpler, well-structured image datasets like MNIST (handwritten digits) or CIFAR-10, FNNs can effectively classify images by treating the pixel values as a flat input vector.
   - **Sentiment Analysis**: They can classify text (e.g., a movie review) as positive, negative, or neutral based on word frequency or embeddings.
   - **Spam Detection**: FNNs can analyze the features of an email (e.g., word occurrences, sender information) to classify it as spam or not spam.
2. **Regression and Predictive Modeling**: FNNs are also very effective at predicting continuous numerical values.
   - **Financial Forecasting**: They can be used to predict stock prices, housing prices, or credit scores based on a set of market indicators or financial data.
   - **Demand Forecasting**: Businesses use FNNs to predict future sales or demand for a product based on historical sales data, seasonality, and marketing spend.
   - **Medical Diagnosis**: They can predict health outcomes or the likelihood of a disease based on a patient's medical history and test results.
3. **Pattern Recognition**: At their core, FNNs are pattern recognition machines. They can learn to identify complex patterns in data that are not obvious to humans.
   - **Face Recognition**: While CNNs are now state-of-the-art, early face recognition systems used FNNs to identify faces from images.
   - **Speech Recognition**: They can learn to recognize phonemes and words from audio signals, forming a key component of speech recognition systems.

Modeling the network:

1. Input layer – First Hidden Layer – Second Hidden Layer – Output Layer
2. Inputs in input layer : 28 x 28 = 784
3. First hidden layer : 128 neurons to process 784 inputs with ReLU function
4. Second hidden layer : 64 neurons with ReLU function
5. Output layer : 10 neurons for 10 possible outputs with softmax function

Modules used:

1. Tensorflow, Keras : for modeling the network
2. Matplotlib : for graphical visualization of result

Steps involved:

1. **Data preparation** involves loading the data, splitting the data into training and testing, reshaping the image (no. of samples, height, width and channel), normalizing the pixel values and one-hot encoding of the image labels.
2. **Model building** involves creating a stack of layers input layer, hidden layers and output layer, interconnecting with all the neurons in the final layer (dense).

3. **Training and evaluation** involve choosing the best optimizer, loss function, metrics like accuracy for monitoring. In addition, the model accuracy is plotted during the training over time.

Program:

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers, models
from tensorflow.keras.datasets import mnist
import matplotlib.pyplot as plt
#Load the dataset
(x_train, y_train), (x_test, y_test) = mnist.load_data()
#Normalize the pixel values
x_train = x_train.reshape(-1,28*28).astype('float32')/255
x_test = x_test.reshape(-1,28*28).astype('float32')/255
#Model the network
model = keras.Sequential([
   layers.Dense(128, activation="relu",input_shape=(784,)),
   layers.Dense(64, activation="relu"),
   layers.Dense(10, activation="softmax")
])
#Define the metrics
model.compile(optimizer='adam',
          loss='sparse_categorical_crossentropy',
          metrics=['accuracy'])
#Train and evaluate the model
history = model.fit(x_train, y_train, epochs=5, batch_size=128, validation_split=0.1)
test_loss, test_acc = model.evaluate(x_test, y_test)
print(f"Test accuracy : {test_acc:.4f}")
#Plot the graphs - Accuracy and Loss
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('Model Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend(['Train', 'Validation'], loc='upper left')
plt.show()
```

Output: Test accuracy: 0.9748

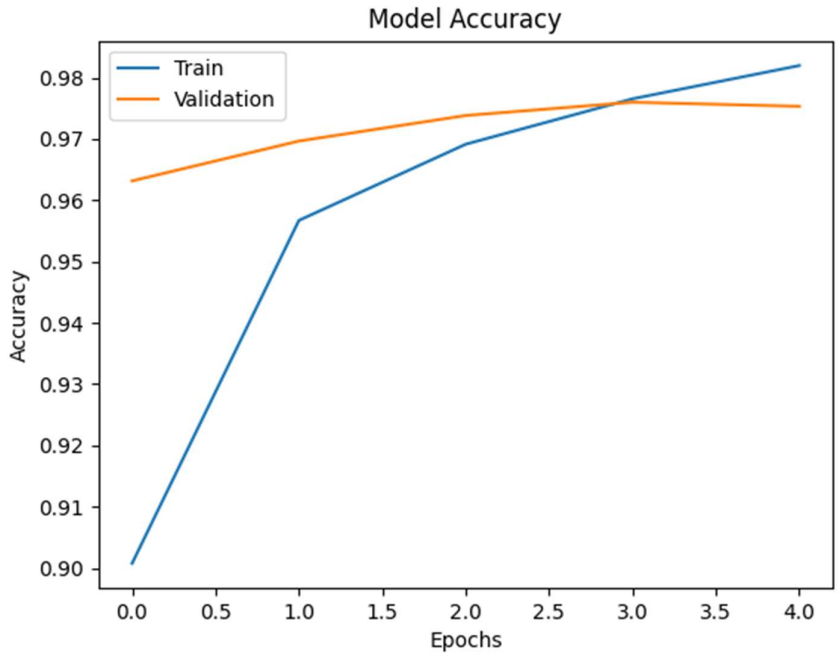


Fig. 1 Model Accuracy plot